



INTRODUCTION AUX COMMUNICATIONS NUMÉRIQUES (P7A-RY)

Tronc commun aux parcours *ComRF* et *ORPHeO*

Laboratoire

Chaîne de transmission numérique

DEUX SÉANCES DE 1 h 15

Canal AWGN, puis canal sélectif en fréquence

Vincent Choqueuse

Rien n'est importé d'une bibliothèque de communications numériques. Le canal vous est donné ; le récepteur — détecteur et égaliseur — est à écrire, et chaque courbe simulée est confrontée à la courbe théorique qui la prédit.

`numpy   scipy   matplotlib`

ENIB, Bretagne INP

Ce que ce laboratoire vise

Ce laboratoire sert quatre des composantes de l'acquis d'apprentissage **AAV2** du module, celles que les interventions couvrent. Le tableau 1 indique où chacune se travaille. Il se lit dans les deux sens : aucune composante n'est sans question, et aucune question ne sort du périmètre.

TABLE 1 – Les quatre composantes de l'AAV2 et les questions qui les traitent.

N°	Composante de l'AAV2	Questions
1	Décrire l'architecture d'une chaîne, relier débit binaire, bande passante et efficacité spectrale	1.1, 1.6, 2.6
2	Choisir et justifier une modulation mono-porteuse (ASK, PSK, QAM) adaptée à un cahier des charges, en explicitant le compromis entre efficacité spectrale et robustesse	1.1, 1.6
3	Représenter et interpréter les signaux modulés (représentation I/Q, constellation), et l'interférence entre symboles	1.1, 1.2, 2.1
4	Modéliser un canal AWGN, employer le rapport signal à bruit et E_b/N_0 pour comparer des schémas de modulation	1.2, 1.3, 1.4, 1.5, 2.2, 2.5

Ce qui est hors périmètre. La mise en forme par filtrage de Nyquist n'est pas implantée : on travaille au rythme symbole, un échantillon par symbole, et la bande occupée n'intervient que dans les calculs d'efficacité spectrale. La modulation de fréquence (FSK) n'entre pas dans le cadre des constellations et n'est pas traitée. Le codage correcteur et le CP-OFDM sont mentionnés en conclusion, sans être mis en œuvre.

Organisation

Deux séances de 1 h 15. La première construit la chaîne complète sur canal gaussien et la valide contre la théorie. La seconde casse l'hypothèse de canal plat et reconstruit un récepteur qui tient.

Environnement. Python 3, avec `numpy`, `scipy` et `matplotlib`. Rien d'autre. Aucune bibliothèque de communications numériques n'est employée : c'est le sujet du laboratoire.

Le squelette. Les *briques* de la chaîne sont dans `code/src/`, un module par fonction du schéma. Ce qui *subit* la transmission vous est donné — le canal, le mapping binaire, les constellations, les compteurs d'erreurs, les courbes théoriques, le tracé — parce que ce n'est pas ce que le laboratoire évalue. Ce qui *décide* est à écrire : le détecteur, la matrice du canal et l'égaliseur. Ces emplacements sont marqués `# TODO`, suivis du numéro de séance et de question.

Les deux *scripts* de séance, à la racine de `code/`, sont **vides** : ils ne contiennent que les titres de leurs cinq sections. C'est vous qui les écrivez, entièrement. Chaque question dit ce que sa section doit produire et donne, sous le titre **À utiliser**, les fonctions disponibles pour l'écrire ; l'assemblage, lui, ne vous est pas donné.

Les cinq sections. Chaque script s'écrit en cinq sections, **A** à **E**, une par question, dans l'ordre où la chaîne transmet. Écrire une section, exécuter le script, vérifier le résultat annoncé, et seulement ensuite passer à la suivante :

```
| python tp1_awgn.py
```

Une section fautive se voit tout de suite ; une section fautive découverte trois sections plus loin coûte vingt minutes.

Trois règles, applicables aux deux séances.

TABLE 2 – Les sept modules de `src/` et les deux scripts de séance. La colonne de droite renvoie à la question qui les remplit.

Fichier	Rôle	Question
<i>src/ — les briques</i>		
<code>mapper.py</code>	bits $\leftrightarrow$ indices $\leftrightarrow$ symboles	<i>fourni</i>
<code>modulators.py</code>	constellations ASK, PSK, QAM; normalisation en énergie	<i>fourni</i>
<code>visualization.py</code>	tracé des constellations, enregistrement des figures	<i>fourni</i>
<code>channel.py</code>	canal AWGN (de variance N_0 donnée) et canal sélectif	<i>fourni</i> , sauf 2.4
<code>detectors.py</code>	projecteur dur (<i>hard projector</i>), égaliseur ZF	1.3, 2.5
<code>metrics.py</code>	SER, BER, précision d'une mesure	<i>fourni</i>
<code>theoretical.py</code>	fonction Q , SER et BER théoriques	<i>fourni</i>
<i>code/ — les deux séances</i>		
<code>tp1_awgn.py</code>	séance 1 : sections A à E, du mapping aux courbes	<i>vide</i>
<code>tp2_selectif.py</code>	séance 2 : sections A à E, du canal à l'égaliseur	<i>vide</i>

1. **Chaque module se teste avant d'être utilisé.** Les sept fichiers de `src/` sont exécutables et portent leur propre contrôle. Ne pas passer à la question suivante tant que, depuis `code/src/`, `python <module>.py` n'affiche pas le résultat annoncé. Une erreur non détectée à ce stade coûtera vingt minutes plus tard.
2. **Toute courbe simulée porte sa courbe théorique.** C'est la seule façon de savoir si le code est juste. Un écart visible est une information, pas une fatalité : il faut l'expliquer.
3. **Tout point mesuré repose sur assez d'erreurs.** La règle des cent erreurs du §1.7 du polycopié : l'écart-type relatif d'une estimation de Monte-Carlo vaut $1/\sqrt{N_{\text{err}}}$. C'est le nombre d'erreurs, et non celui des symboles, qui décide de la précision ; les boucles de simulation s'arrêtent donc sur un quota d'erreurs.

Conventions du code. Elles sont posées par les deux modules fournis et valent partout ensuite.

- **Énergie moyenne unitaire** : tout alphabet vérifie $\mathbb{E}[|x[n]|^2] = 1$. Sans cette normalisation, comparer deux constellations n'aurait aucun sens.
- **Bruit complexe circulaire** : $\mathbb{E}[|w[n]|^2] = N_0$, soit $N_0/2$ sur chacune des voies I et Q.
- **Le détecteur rend des indices**, jamais des symboles ni des bits. C'est `indices_to_bits` qui termine le travail.
- **Graine fixée** : chaque étape crée son générateur par `np.random.default_rng(SEED)`. Deux exécutions donnent la même figure.
- **Les constellations passent par `visualization.py`** : `plot_constellation` construit sa figure, l'affiche et la retourne — option `show`, vraie par défaut, à mettre à `False` pour enchaîner plusieurs figures sans être interrompu — et `save_figure` l'écrit dans `figures/`. Les **courbes de taux d'erreur**, elles, se tracent directement avec `matplotlib` dans le script : `import matplotlib.pyplot as plt`, puis un `ax.semilogy` par courbe.
- **Aucune boucle sur les échantillons.** Les seules boucles admises portent sur les points d'une courbe, les paquets d'une simulation ou les colonnes d'une matrice construite une fois pour toutes.

1. Séance 1 : le canal AWGN

La chaîne à construire est celle de la figure 1.1 du polycopié, réduite à ce qui compte ici :

$$\underbrace{b[n]}_{\text{bits}} \longrightarrow \underbrace{x[n] \in \mathcal{M}}_{\text{mapping}} \longrightarrow \underbrace{y[n] = x[n] + w[n]}_{\text{canal AWGN}} \longrightarrow \underbrace{\hat{x}[n]}_{\text{décision}} \longrightarrow \underbrace{\hat{b}[n]}_{\text{demapping}} \longrightarrow \text{BER}$$

Les quatre modulations comparées de la section B à la section E sont la **BPSK**, la **QPSK**, la **8-PSK** et la **16-QAM** : une par famille pour les deux premières, deux tailles pour voir ce que coûte un alphabet plus dense. La modulation de travail, celle sur laquelle s'écrivent les sections A à D, est la QPSK.

Question 1.1 — les constellations, et le mapping

15 min

`src/modulators.py` est fourni : les trois familles du chapitre 3 du polycopié y sont écrites — M points réels équirépartis (ASK), M points sur le cercle unité (PSK), $\sqrt{M} \times \sqrt{M}$ points en grille carrée (QAM) — avec la normalisation $\mathbb{E}[|x|^2] = 1$ et l'étiquetage, qui fait de l'indice d'un point son étiquette binaire. Deux étiquetages sont disponibles, par l'argument `order` de `get_alphabet` : "bin", le binaire naturel, qui est le **défaut**, et "gray", où deux points voisins ne diffèrent que d'un seul bit. *Les points sont les mêmes ; seule leur numérotation change* — la question 1.4 en fait la mesure. Le lire, puis exécuter son contrôle depuis `code/src/` :

```
| python modulators.py
```

La normalisation n'est pas un détail de présentation : sans elle, il suffirait d'écarter les points pour paraître plus robuste, et aucune comparaison entre modulations n'aurait de sens. Tout s'obtient ensuite par un seul appel, `get_alphabet(famille, M, order)`.

PIÈGE. Toute la chaîne du laboratoire travaille en `order="gray"`, et il faut donc le demander : c'est l'étiquetage de Gray, et lui seul, qui autorise $\text{BER} = \text{SER}/k$ et rend les courbes de la section E comparables à la théorie. L'oublier ne provoque aucune erreur : le SER reste juste, et seul le BER s'écarte — d'environ un tiers en 16-QAM, soit un demi-décibel, qu'on peut mettre longtemps sur le compte du hasard. Fixer l'ordre une fois, en tête de script, et l'utiliser partout.

Ouvrir `tp1_awgn.py` : il est vide. Écrire la **section A**, qui doit produire :

- les quatre constellations de la figure 1, engendrées et étiquetées : la 4-PAM, la 4-QAM, la 16-QAM et la 64-QAM. Une figure par constellation, `figures/tp1-alphabet-4pam.pdf` et les trois autres ;
- une trame de bits équiprobables, les indices qu'ils forment et les symboles QPSK correspondants, les trois affichés sur quelques symboles ;
- le contrôle que le chemin inverse redonne exactement les bits de départ.

Le mapping se fait en deux temps, qu'il vaut mieux ne pas confondre :

$$\underbrace{b[n] \in \{0, 1\}}_{\text{bits}} \longrightarrow \underbrace{i[n] \in \{0, \dots, M-1\}}_{\text{indices}} \longrightarrow \underbrace{x[n] \in \mathcal{M}}_{\text{symboles}}$$

L'indice est l'étiquette binaire lue comme un entier, bit de poids fort en tête ; c'est lui, et non les bits, que le détecteur rendra à la question 1.3.

À utiliser. `modulators` : `get_alphabet` (argument `order`), `bits_per_symbol`. `mapper` : `generate_bits`, `bits_to_indices`, `map_to_symbols`, `indices_to_bits`. `visualization` : `plot_constellation` (options `labels` et `show`), qui affiche sa figure et la retourne, puis `save_figure` pour l'écrire.

- (a) Combien de bits faut-il générer pour obtenir n symboles ?

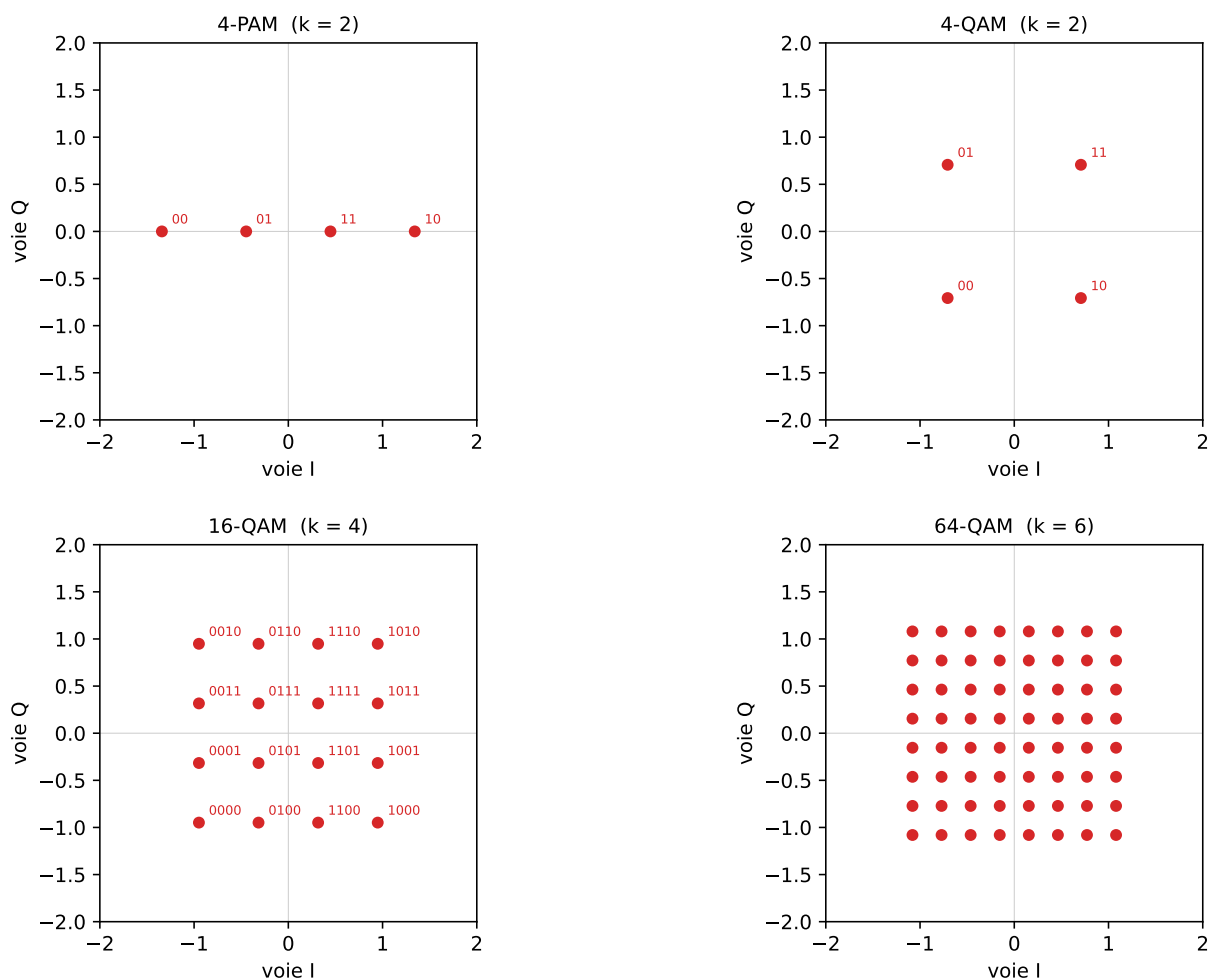


FIGURE 1 – Les quatre constellations à reproduire. Les quatre axes ont la même échelle, et les quatre alphabets la même énergie moyenne : c'est pourquoi les points se resserrent quand M augmente. Au-delà de 16 points les étiquettes ne sont plus lisibles, et la 64-QAM est tracée sans elles.

- (b) Sur la 16-QAM, prendre deux points voisins et vérifier sur leurs étiquettes qu'un seul bit change.
- (c) Relever k et $d_{\min}$ dans la sortie de `modulators.py`, et donner en une phrase le compromis entre les deux.

VÉRIFICATION. Le contrôle aller-retour passe, et les quatre figures reproduisent celles de la figure 1. `python modulators.py` affiche $\mathbb{E}[|x|^2] = 1,000000$ et les distances minimales 2,0000 ; 1,4142 ; 0,7654 ; 0,6325.

Question 1.2 — le canal AWGN, et la variance du bruit

10 min

`src/channel.py` est fourni : le canal est le milieu que l'on subit, pas ce que ce laboratoire fait construire. Sa fonction `awgn(x, n0, rng)` ajoute au signal un bruit complexe circulaire de variance N_0 , soit $N_0/2$ sur chacune des voies I et Q. Elle attend cette **variance**, et non un rapport en décibels : la conversion est à vous.

Le rapport signal à bruit *par bit* s'y ramène en une ligne. L'énergie par bit vaut $E_b = E_s/k$, d'où

$$N_0 = \frac{E_b}{E_b/N_0} = \frac{E_s}{k \cdot 10^{(E_b/N_0)_{\text{dB}}/10}}, \quad E_s = 1 \text{ (alphabets normalisés)}.$$

Écrire la **section B** : cette conversion, puis le signal reçu $y = x + w$, et les quatre modulations de la séance — BPSK, QPSK, 8-PSK, 16-QAM — observées à la sortie du canal à 8 puis à 15 dB, dans `figures/tp1-awgn-qpsk-08db.pdf` et les sept autres — une figure par modulation et par rapport.

À utiliser. `channel : awgn`, `visualization : plot_constellation` (paramètre `y`, le nuage observé), `save_figure`. Plus les fonctions de mapping de la question précédente.

- (a) Vérifier la conversion : tirer le bruit seul, et comparer sa variance mesurée $\mathbb{E}[|w|^2]$ au N_0 calculé, à 8 puis à 15 dB.
- (b) À 8 dB, quels nuages commencent à se toucher ? Rapprocher de la colonne $d_{\min}$ affichée par `python modulators.py`.

PIÈGE. La division par k est ce qui rend les modulations comparables. L'oublier décalerait toute une courbe de $10 \log_{10} k$ décibels, soit 6 dB pour la 16-QAM, sans lui ôter son allure crédible. `awgn` accepte sans broncher n'importe quel nombre positif : c'est le contrôle de (a), et lui seul, qui vous dira que la variance est la bonne.

VÉRIFICATION. `python channel.py` affiche un N_0 mesuré égal au N_0 demandé à la troisième décimale, et en section B la variance mesurée retombe sur la valeur calculée : 0,079 à 8 dB et 0,016 à 15 dB, en QPSK.

Question 1.3 — le projecteur dur, et le SER

10 min

Le récepteur décide, pour chaque observation, le point de la constellation le plus proche : c'est le *projecteur dur* (*hard projector*). Compléter `src/detectors.py`, fonction `hard_projector` :

$$\hat{t}[n] = \arg \min_i |y[n] - \mathcal{M}_i|^2,$$

règle optimale sur le canal AWGN à symboles équiprobables (poly, §2.2) : le maximum a posteriori s'y ramène au maximum de vraisemblance, puis, le bruit étant gaussien, à une minimisation de distance. **Sans boucle sur les échantillons** : la diffusion de `numpy` compare d'un coup toutes les observations à tous les points de l'alphabet. La fonction rend des **indices**, jamais des symboles.

Deux modules accompagnent la mesure, et sont fournis. `src/metrics.py` compte les symboles faux et les bits faux *séparément*, ce qui permet de vérifier la relation entre les deux au lieu de la supposer. `src/theoretical.py` transcrit les résultats admis du polycopié, déjà écrits en fonction de E_b/N_0 ; les

lire avant de s'en servir, c'est à eux que toute mesure sera comparée :

$$Q(x) = \frac{1}{2} \operatorname{erfc}(x/\sqrt{2}) \quad \text{scipy.special.erfc}$$

$$\operatorname{SER}_{\text{ASK}} \approx 2 \left(1 - \frac{1}{M}\right) Q\left(\sqrt{\frac{6k}{M^2-1} \frac{E_b}{N_0}}\right) \quad \text{poly, \acute{e}q. (3.4)}$$

$$\operatorname{SER}_{\text{PSK}} \approx 2 Q\left(\sin\left(\frac{\pi}{M}\right) \sqrt{2k \frac{E_b}{N_0}}\right) \quad \text{poly, \acute{e}q. (3.5)}$$

$$\operatorname{SER}_{\text{QAM}} \approx 4 \left(1 - \frac{1}{\sqrt{M}}\right) Q\left(\sqrt{\frac{3k}{M-1} \frac{E_b}{N_0}}\right) \quad \text{poly, \acute{e}q. (3.6)}$$

Écrire la **section C** : décider, puis compter. D'abord *sans bruit*, où la décision doit être exacte ; puis sur une quarantaine de symboles à $E_b/N_0 = 0$ dB, où les indices émis et décidés se comparent à l'œil ; enfin le SER mesuré sur 200 000 symboles à $E_b/N_0 = 6$ dB, confronté au SER théorique au même E_b/N_0 .

À utiliser. `detectors : hard_projector.metrics : count_symbol_errors, compute_ser.theoretical : ser_theory`. Plus la chaîne des sections A et B.

- (a) Relever le SER mesuré et le SER théorique. L'écart relatif est-il compatible avec la précision $1/\sqrt{N_{\text{err}}}$ que donne le nombre d'erreurs comptées ?

PIÈGE. Le tableau des distances occupe M fois la mémoire du signal, soit 256 Mo pour un million de symboles en 16-QAM. D'où le traitement par paquets de la question 1.5.

VÉRIFICATION. `python detectors.py` annonce un projecteur exact sans bruit ; `python theoretical.py` donne, à 6 dB, les SER théoriques 0,00239 (BPSK), 0,00478 (QPSK), 0,06144 (8-PSK) et 0,11149 (16-QAM). En section C, la QPSK mesure $\operatorname{SER} \approx 0,0048$ sur 200 000 symboles. Les contrôles annoncés comme non écrits (ZF, BER théorique) sont normaux à ce stade.

Question 1.4 — du symbole au bit : le BER

10 min

Le projecteur rend des indices ; ce qui intéresse l'utilisateur, ce sont des bits. `indices_to_bits` termine le démappage : chaque indice se réécrit sur k bits, bit de poids fort en tête. C'est exactement l'opération inverse de `bits_to_indices` de la section A, et c'est l'étiquetage de Gray qui fait que les deux comptages ne donnent pas la même chose.

Du côté de la théorie, `ber_theory` ne fait que diviser :

$$\operatorname{BER} \approx \frac{\operatorname{SER}}{k} \quad (\text{poly, prop. 3.4}).$$

L'approximation tient parce que, sous étiquetage de Gray, une erreur entre plus proches voisins ne fausse qu'un seul des k bits du symbole. C'est précisément ce que (b) fait mettre en défaut.

Écrire la **section D** : sur la trame de la section C, le nombre de bits faux, le BER mesuré, et sa comparaison à SER/k et à la valeur théorique.

À utiliser. `mapper : indices_to_bits.metrics : count_bit_errors, compute_ber.theoretical : ber_theory`.

- (a) Comparer le BER mesuré, SER/k et le BER théorique. Les trois coïncident-ils ?
 (b) Refaire la mesure en 16-QAM à 12 dB, une fois avec `order="gray"`, une fois avec `order="bin"`. Relever les deux SER et les deux BER, et dire lequel des deux dépend de l'étiquetage.

VÉRIFICATION. En section D, la QPSK à 6 dB donne $\operatorname{BER} \approx 0,00239$, soit exactement SER/k et la valeur théorique. En 16-QAM à 12 dB, les deux étiquetages donnent le même $\operatorname{SER} \approx 5,6 \cdot 10^{-4}$, mais $\operatorname{BER} = 1,4 \cdot 10^{-4}$ en Gray contre $1,9 \cdot 10^{-4}$ en binaire ; `python modulators.py` chiffre la cause, 1,00 bit de différence entre voisins contre

1,33. `python theoretical.py` reproduit alors le tableau 3.1 du polycopié : 9,6; 9,6; 13,0; 13,4; 17,8; 22,5 dB. Si une seule valeur diffère, la formule correspondante est fautive ; ne pas poursuivre.

Question 1.5 — la simulation de Monte-Carlo

15 min

Un point mesuré ne dit rien d'une chaîne : il faut la courbe entière. Écrire la **section E**, la dernière : la chaîne des sections A à D, répétée sur toute la grille E_b/N_0 , et confrontée à la théorie.

Pour chaque modulation et chaque point de la grille, accumuler des paquets de 10^5 symboles tant que *les deux conditions suivantes* tiennent :

$$N_{\text{err}} < 200 \quad \text{et} \quad N_{\text{sym}} < 2 \cdot 10^6.$$

Le taux mesuré est le rapport des totaux. C'est la première condition qui fixe la qualité de la mesure : s'arrêter sur un quota d'*erreurs*, et non sur un nombre de symboles, donne à tous les points la même précision relative, $1/\sqrt{200} \approx 7\%$. Le paquet, lui, borne la mémoire.

PIÈGE. La seconde condition n'est pas un détail de performance. À 16 dB, une QPSK peut ne produire *aucune* erreur : une boucle qui attend seulement ses 200 erreurs ne s'arrête jamais. Toute boucle d'accumulation porte une borne sur le nombre de symboles.

Tracer enfin la courbe simulée et sa courbe théorique dans `figures/tp1-ber.pdf`, et afficher pour chaque modulation le E_b/N_0 requis et l'efficacité spectrale.

À utiliser. `theoretical : ber_theory, required_ebn0`. `modulators : spectral_efficiency`. `matplotlib : plt.subplots, ax.semilogy` (une courbe théorique en trait, une simulation en points), `ax.grid, ax.legend`. `visualization : save_figure`.

- Vérifier que les points tombent sur les courbes théoriques. Un décalage de 3 ou 6 dB sur une seule modulation désigne une erreur précise : laquelle ?
- Vérifier que la BPSK et la QPSK ont la même courbe (poly, §3.5).
- Combien de bits faut-il simuler pour mesurer un BER de 10^{-5} avec 200 erreurs ? Combien de temps, au rythme de la section E ?

VÉRIFICATION. `figures/tp1-ber.pdf`, où les points simulés sont sur les courbes théoriques.

Question 1.6 — trancher un cahier des charges

5 min

Cahier des charges du cours : 10 Mbit/s dans 5 MHz, BER inférieur à 10^{-5} , canal offrant $E_b/N_0 = 14$ dB, $\alpha = 0,35$.

La console de la section E donne pour chaque modulation le E_b/N_0 requis et l'efficacité spectrale $\eta = \log_2 M / (1 + \alpha)$. En dresser le tableau, éliminer les modulations qui échouent, nommer celle qui est retenue et chiffrer sa marge en décibels.

VÉRIFICATION. Les figures de la séance — les quatre constellations, les huit constellations bruitées et `figures/tp1-ber.pdf` —, plus le tableau du cahier des charges. Une modulation nommée, un budget vérifié, une marge chiffrée.

2. Séance 2 : le canal sélectif en fréquence

Le canal n'ajoute plus seulement du bruit : il mélange les symboles. Chaque observation en contient plusieurs (poly, déf. 4.2) :

$$y[n] = \sum_{l=0}^{L_h-1} h[l] x[n-l] + w[n].$$

Toute la séance consiste à mesurer le dégât, à constater que le récepteur optimal est hors de portée, puis à en construire un qui soit calculable.

La modulation est la QPSK, la même qu'en séance 1, et le canal a trois trajets. `tp2_selectif.py` est vide, comme celui de la séance 1, et s'écrit en cinq sections A à E. Dans les sections A et B, l'émission est un flux continu de symboles ; à partir de la section D, elle se fait par **blocs** de $N = 64$ symboles séparés par un intervalle de garde de $L_h - 1$ symboles nuls, de sorte que chaque bloc traverse le canal sans interférer avec le suivant.

Question 2.1 — le canal, et ce qu'il fait à la constellation

10 min

Le canal sélectif est fourni, comme celui de la séance 1 : lire `apply_channel` et `normalize_channel` dans `src/channel.py`. La convolution est complète, de longueur $N + L_h - 1$: la traîne n'est pas tronquée, elle porte de l'information et la question 2.5 s'en sert. La normalisation $\sum_l |h[l]|^2 = 1$ garantit que le canal ne crée ni ne détruit de puissance, de sorte que le E_b/N_0 en abscisse reste celui de la séance 1.

Écrire la **section A** : la réponse impulsionnelle du canal $[1, 0.5, 0.3]$ normalisée, et la constellation reçue **sans aucun bruit** : `figures/tp2-canal.pdf` pour la première, `figures/tp2-isi.pdf` pour la seconde.

À utiliser. `channel : normalize_channel, apply_channel, visualization : plot_impulse_response, plot_constellation` (options `lim` et `frontieres`), `save_figure`.

- (a) Compter les nuages par point de la QPSK, et retrouver ce nombre à partir de M et L_h .

VÉRIFICATION. Le nuage de `tp2-isi.pdf` est tracé sans bruit : tout ce qu'on y voit est de l'interférence. Les deux traits bleus sont les frontières de décision de la QPSK.

Question 2.2 — le récepteur de la séance 1, appliqué tel quel

15 min

Écrire la **section B** : un flux continu de symboles, le canal, le bruit, puis le récepteur de la séance 1 appliqué sans rien changer. Ce récepteur ignore l'interférence : il corrige le gain du trajet principal, en divisant par $h[0]$, et décide symbole par symbole. La boucle de Monte-Carlo est celle de la question 1.5, en SER cette fois. Produire `figures/tp2-ber-naif.pdf`, avec la courbe du canal plat en référence.

Le critère du plancher d'erreur (poly, prop. 4.2), fourni dans `channel.py`, dit à quoi s'attendre :

$$\sum_{l \neq 0} |h[l]| < |h[0]| \implies \text{perte fixe,} \quad \text{sinon} \quad \text{plancher d'erreur.}$$

À utiliser. `channel : error_floor_criterion, apply_channel, awgn` (qui attend toujours la variance N_0 : reprendre la conversion de la question 1.2, valable ici parce que le canal est normalisé). `detectors : hard_projector, metrics : count_symbol_errors, compute_ser, theoretical : ser_theory`. Plus les fonctions de tracé de la question 1.5.

- (a) Évaluer le critère pour $[1, 0.5, 0.3]$. Que prédit-il ? Mesurer sur la figure, à $\text{SER} = 10^{-2}$, l'écart en décibels avec le canal plat. Est-ce compatible ?
- (b) Reprendre avec le canal $[1, 0.8, 0.6]$, où le critère est renversé. Que devient la courbe ? Rétablir ensuite le canal initial.

PIÈGE. Le plancher n'est pas une propriété du *canal*, c'est le prix d'un *récepteur* qui ignore l'interférence : la suite le montre en construisant un meilleur récepteur sur le même canal.

Question 2.3 — le récepteur optimal, et pourquoi on ne l'écrira pas

5 min

Le critère du maximum de vraisemblance ne change pas de nature, mais il porte désormais sur la *séquence*

entière (poly, éq. 4.5) :

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x} \in \mathcal{M}^N} \sum_n \left| y[n] - \sum_{l=0}^{L_h-1} h[l] x[n-l] \right|^2.$$

Écrire la **section C**, qui n'est qu'un calcul d'ordre de grandeur.

À utiliser. `detectors : ml_sequence_cost`, fournie.

- Combien de séquences candidates pour une QPSK et un bloc de 64 symboles ? Retrouver ce nombre à partir de M et de N .
- À 10^9 candidats par seconde, combien de temps pour détecter *un seul* bloc ? Le comparer à l'âge de l'univers, $4,4 \cdot 10^{17}$ s.

Question 2.4 — le canal, écrit comme une matrice

20 min

Compléter `src/channel.py`, fonction `convolution_matrix`.

La convolution est linéaire : sur un bloc de N symboles, elle s'écrit comme un produit matriciel. La colonne n de $\mathbf{H}$, de taille $(N + L_h - 1) \times N$, porte la réponse impulsionnelle décalée de n : c'est la trace laissée par $x[n]$ dans l'observation.

$$\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{w}, \quad \mathbf{H} = \begin{pmatrix} h[0] & 0 & \cdots & 0 \\ h[1] & h[0] & & \vdots \\ \vdots & h[1] & \ddots & 0 \\ h[L_h - 1] & \vdots & & h[0] \\ 0 & h[L_h - 1] & & h[1] \\ \vdots & & \ddots & \vdots \\ 0 & \cdots & 0 & h[L_h - 1] \end{pmatrix}$$

Elle est constante le long de ses diagonales : c'est une matrice de **Toeplitz**, et cette structure n'est rien d'autre que l'invariance du canal dans le temps.

Écrire la **section D** : construire $\mathbf{H}$, puis *vérifier qu'elle décrit bien le canal de la section A*, c'est-à-dire que $\mathbf{H}\mathbf{x}$ et la convolution coïncident à 10^{-15} près sur un vecteur quelconque. Tant que ce contrôle ne passe pas, la suite est inutile.

À utiliser. `channel : convolution_matrix` (celle que vous venez d'écrire), `apply_channel`. `numpy : np.linalg.norm`, et l'opérateur `@` pour le produit matriciel.

- Relever la taille de $\mathbf{H}$ pour $N = 64$ et $L_h = 3$, et dire d'où viennent ses $L_h - 1$ lignes supplémentaires.
- Les blocs étant rangés en colonnes dans une matrice $\mathbf{X}$, combien de produits matriciels faut-il pour transmettre mille blocs ?

PIÈGE. Tronquer la sortie à N échantillons reviendrait à jeter la traîne du canal, donc de l'information ; l'égaliseur de la question suivante s'en sert. C'est aussi pourquoi l'intervalle de garde dure $L_h - 1$ symboles.

Question 2.5 — l'égaliseur à forçage à zéro

20 min

Compléter `src/detectors.py`, fonctions `zf_equalizer` et `noise_enhancement`. L'égaliseur inverse le canal au sens des moindres carrés :

$$\hat{\mathbf{x}} = (\mathbf{H}^H \mathbf{H})^{-1} \mathbf{H}^H \mathbf{y} = \mathbf{H}^+ \mathbf{y}.$$

Le nom vient de ce que l'interférence est annulée *exactement* : sans bruit, $\mathbf{H}^+ \mathbf{H} \mathbf{x} = \mathbf{x}$. La décision de la séance 1 s'applique alors telle quelle.

Écrire la **section E** : l'émission par blocs, $\mathbf{Y} = \mathbf{H}\mathbf{X} + \mathbf{W}$, l'égalisation, la décision, et la figure figures/tp2-ber.pdf portant les trois courbes — canal plat théorique, récepteur naïf, égaliseur ZF.

À utiliser. detectors : zf_equalizer, noise_enhancement, hard_projector. channel : convolution_matrix, awgn. numpy : np.linalg.pinv pour la pseudo-inverse. Les courbes du naïf sont celles de la section B, déjà calculées.

- (a) Mesurer sur la figure, à $\text{SER} = 10^{-3}$, l'écart en décibels entre le ZF et le canal plat, puis le comparer à $10 \log_{10}$ du facteur rendu par noise_enhancement. Les deux valeurs doivent coïncider.

PIÈGE. Forcer l'interférence à zéro et minimiser l'erreur ne sont pas la même chose : le ZF fait le premier, et paie le second. C'est le point de départ de l'égaliseur MMSE, hors périmètre ici.

VÉRIFICATION. python detectors.py annonce une erreur du ZF sans bruit de l'ordre de 10^{-15} et une amplification du bruit de 1,712, soit 2,34 dB.

Question 2.6 — ce que la solution coûte

5 min

L'intervalle de garde de $L_h - 1$ symboles est du temps pendant lequel rien n'est transmis.

- (a) Exprimer l'efficacité spectrale utile en fonction de N , L_h et η , chiffrer la perte pour $N = 64$ et $L_h = 3$, puis pour $N = 16$.

VÉRIFICATION. Les quatre figures de la séance, dans figures/ : tp2-canal.pdf, tp2-isi.pdf, tp2-ber-naïf.pdf et tp2-ber.pdf ; et les réponses écrites aux questions 2.1 à 2.6.

Livrable

Un répertoire par binôme, contenant :

1. les deux modules de src/ complétés et les deux scripts de séance écrits, chacun passant son propre test ;
2. les figures produites, dans figures/ ;
3. un compte rendu de deux pages au plus, répondant aux questions posées et comportant le tableau du cahier des charges de la question 1.6.

Ce qui est évalué. Les figures d'abord : une courbe simulée sans sa courbe théorique ne compte pas. Le code ensuite : graine fixée, opérations vectorisées, axes légendés avec leurs unités. Les réponses écrites enfin, où l'on attend des chiffres et non des impressions.

Le fil des deux séances, en une phrase. La séance 1 établit qu'une chaîne bien construite atteint exactement la performance que la théorie prédit ; la séance 2 casse l'hypothèse qui rendait cette théorie valable, et montre ce qu'il en coûte de la rétablir.

Allure attendue des deux figures de sortie

Ces deux figures sont celles que le code complété doit produire. Elles sont données pour que vous sachiez immédiatement si une valeur est fautive : une courbe décalée de quelques décibels, ou qui ne rejoint pas sa théorie, désigne une erreur précise, et le sujet indique laquelle. Les mesures et les commentaires demandés restent à faire.

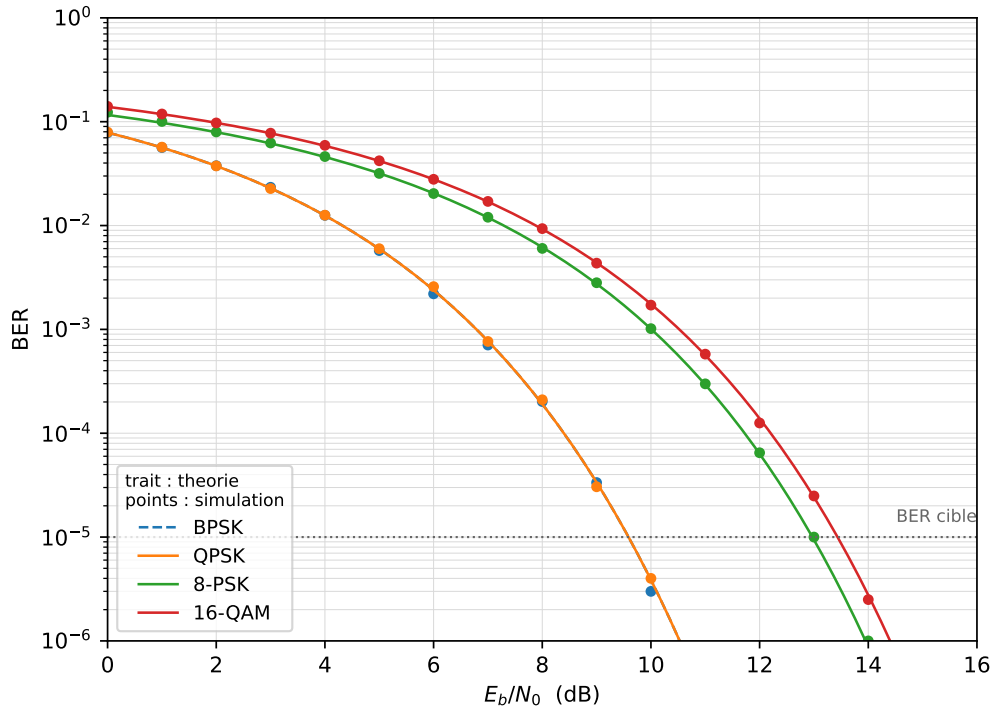


FIGURE 2 – Séance 1, question 1.5. Les points simulés tombent sur les courbes théoriques pour les quatre modulations. La BPSK, en trait discontinu, est confondue avec la QPSK ; ses points s’arrêtent plus tôt, parce qu’à nombre de symboles égal elle produit deux fois moins de bits, donc deux fois moins d’erreurs à compter.

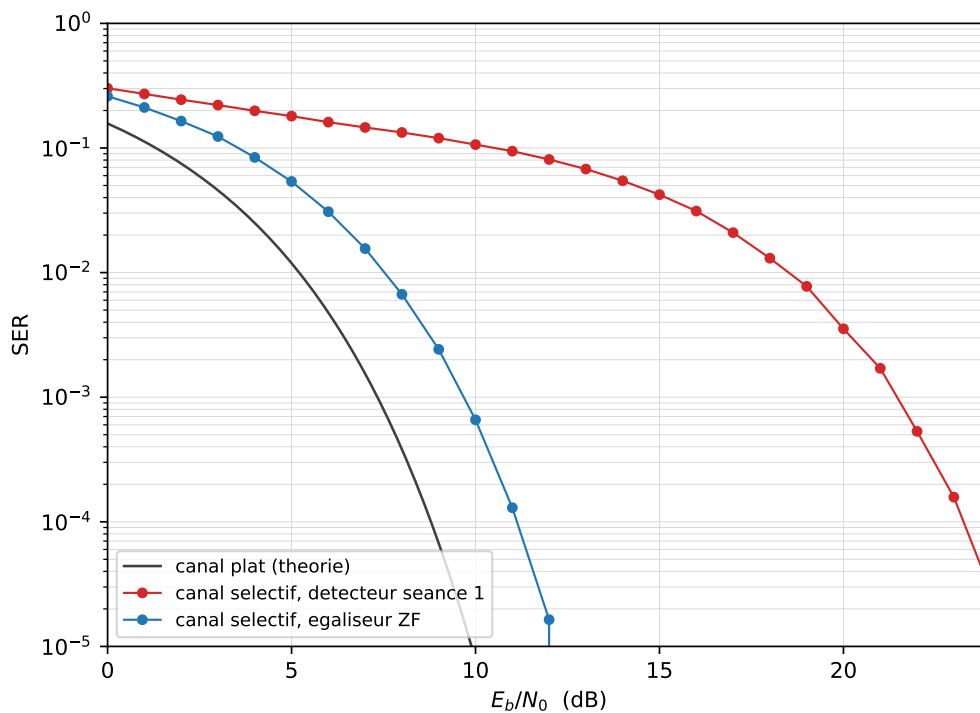


FIGURE 3 – Séance 2, question 2.5. Le détecteur de la séance 1, appliqué tel quel sur le canal sélectif, perd plus de dix décibels. L’égaliseur ZF les récupère presque tous : il ne reste que l’amplification du bruit, dont la question 2.5(b) demande la valeur exacte.